

Hierarchical and Network Models

The **hierarchical model** was developed in the 1960s to manage large amounts of data for complex manufacturing projects, such as the Apollo rocket that landed on the moon in 1969. The model's basic logical structure is represented by an upside-down tree. The hierarchical structure contains levels, or segments. A **segment** is the equivalent of a file system's record type. Within the hierarchy, a higher layer is perceived as the parent of the segment directly beneath it, which is called the child. The hierarchical model depicts a set of one-to-many (1:M) relationships between a parent and its children segments. (Each parent can have many children, but each child has only one parent.)

The **network model** was created to represent complex data relationships more effectively than the hierarchical model, to improve database performance, and to impose a database standard. In the network model, the user perceives the network database as a collection of records in 1:M relationships. However, unlike the hierarchical model, the network model allows a record to have more than one parent. While the network database model is generally not used today, the definitions of standard database *concepts* that emerged with the network model are still used by modern data models:

- The **schema** is the conceptual organization of the entire database as viewed by the database administrator.
- The **subschema** defines the portion of the database “seen” by the application programs that actually produce the desired information from the data within the database.
- A **data manipulation language (DML)** defines the environment in which data can be managed and is used to work with the data in the database.
- A **schema data definition language (DDL)** enables the database administrator to define the schema components.

The Relational Model

The **relational model** was introduced in 1970 by E. F. Codd of IBM in his landmark paper “A Relational Model of Data for Large Shared Databanks” (*Communications of the ACM*, June 1970, pp. 377–387). The relational model represented a major breakthrough for both users and designers. To use an analogy, the relational model produced an “automatic transmission” database to replace the “standard transmission” databases that preceded it. Its conceptual simplicity set the stage for a genuine database revolution.

The relational model's foundation is a mathematical concept known as a relation. To avoid the complexity of abstract mathematical theory, you can think of a **relation** (sometimes called a **table**) as a two-dimensional structure composed of intersecting rows and columns. Each row in a relation is called a **tuple**. Each column represents an attribute. The relational model also describes a precise set of data manipulation constructs based on advanced mathematical concepts.

In 1970, Codd's work was considered ingenious but impractical. The relational model's conceptual simplicity was bought at the expense of computer overhead; computers at that time lacked the power to implement the relational model. Fortunately, computer power grew exponentially, as did operating system efficiency. Better yet, the cost of computers diminished rapidly as their power grew.

The relational data model is implemented through a very sophisticated **relational database management system (RDBMS)**. Arguably the most important advantage of the RDBMS is its ability to hide the complexities of the relational model from the user. The RDBMS manages all of the physical details, while the user sees the relational database as a collection of tables

in which data is stored. The user can manipulate and query the data in a way that seems intuitive and logical. Tables are related to each other through the sharing of a common attribute (a value in a column). For example, the CUSTOMER table in Figure below might contain a sales agent's number that is also contained in the AGENT table.

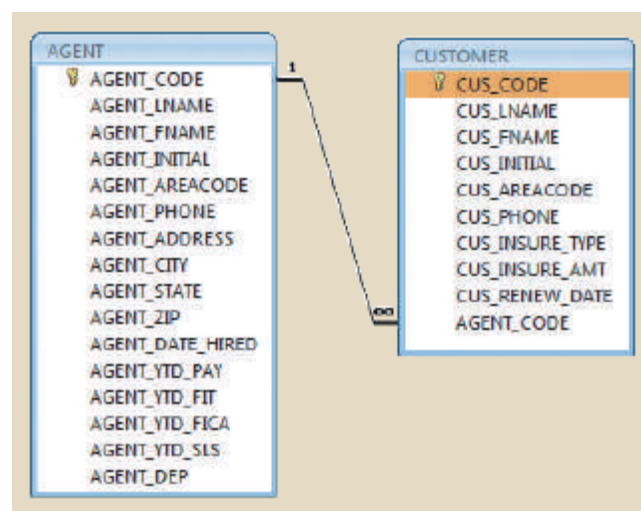
Table name: AGENT (first six attributes)						Database name: Ch02_InsureCo				
AGENT_CODE	AGENT_LNAME	AGENT_FNAME	AGENT_INITIAL	AGENT_AREACODE	AGENT_PHONE					
501	Alby	Alex	B	713	228-1249					
502	Hahn	Leah	F	615	862-1244					
503	Okon	John	T	615	123-5569					

Link through AGENT_CODE

Table name: CUSTOMER										
CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_INSURE_TYPE	CUS_INSURE_AMT	CUS_RENEW_DATE	AGENT_CODE	
10010	Ramas	Alfred	A	615	844-2573	T1	100.00	05-Apr-2018	502	
10011	Dunne	Leona	K	713	894-1238	T1	250.00	16-Jun-2018	501	
10012	Smith	Kathy	W	615	894-2295	S2	150.00	29-Jan-2019	502	
10013	Olowski	Paul	F	615	894-2180	S1	300.00	14-Oct-2018	502	
10014	Orlando	Myron		615	222-1872	T1	100.00	29-Dec-2019	501	
10015	O'Brien	Amy	B	713	442-3381	T2	850.00	22-Sep-2018	503	
10016	Brown	James	G	615	297-1229	S1	120.00	25-Mar-2019	502	
10017	Williams	George		615	290-2558	S1	250.00	17-Jul-2018	503	
10018	Farriss	Anne	G	713	362-7185	T2	100.00	03-Dec-2016	501	
10019	Smith	Olette	K	615	297-3809	S2	500.00	14-Mar-2019	503	

The relational model provides a minimum level of controlled redundancy to eliminate most of the redundancies commonly found in file systems.

The relationship type (1:1, 1:M, or M:N) is often shown in a relational schema, an example of which is shown in Figure below. A [relational diagram](#) is a representation of the relational database's entities, the attributes within those entities, and the relationships between those entities.



Most relational database software uses Structured Query Language (SQL), which allows the user to specify what must be done without specifying how. The RDBMS uses SQL to translate user queries into instructions for retrieving the requested data.

From an end-user perspective, any SQL-based relational database application involves three parts: a user interface, a set of tables stored in the database, and the "SQL engine"

- *The end-user interface.* Basically, the interface allows the end user to interact with the data (by automatically generating SQL code).

- *A collection of tables stored in the database.* In a relational database, all data is perceived to be stored in tables. The tables simply “present” the data to the end user in a way that is easy to understand. Each table is independent. Rows in different tables are related by common values in common attributes.
- *SQL engine.* Largely hidden from the end user, the SQL engine executes all queries, or data requests. Keep in mind that the SQL engine is part of the DBMS software.

The Object-Oriented Model

Increasingly complex real-world problems demonstrated a need for a data model that more closely represented the real world. In the [object-oriented data model \(OODM\)](#), both data *and its relationships* are contained in a single structure known as an [object](#). In turn, the OODM is the basis for the [object-oriented database management system \(OODBMS\)](#). An OODM reflects a very different way to define and use entities. Like the relational model’s entity, an object is described by its factual content. But, quite *unlike* an entity, an object includes information about relationships between the facts within the object, as well as information about its relationships with other objects. Therefore, the facts within the object are given greater *meaning*. The OODM is said to be a [semantic data model](#) because *semantic* indicates meaning.

Because objects include data, various types of relationships, and operational procedures, the object becomes self-contained, thus making it— at least potentially—a basic building block for autonomous structures.

The OO data model is based on the following components:

- An object is an abstraction of a real-world entity. In general terms, an object may be considered equivalent to an ER model’s entity. More precisely, an object represents only one occurrence of an entity.
- Attributes describe the properties of an object. For example, a PERSON object includes the attributes Name, Social Security Number, and Date of Birth.
- Objects that share similar characteristics are grouped in classes. A [class](#) is a collection of similar objects with shared structure (attributes) and behavior (methods). In a general sense, a class resembles the ER model’s *entity* set. However, a class is different from an entity set in that it contains a set of procedures known as *methods*.
- Classes are organized in a *class hierarchy*. The [class hierarchy](#) resembles an upside-down tree in which each class has only one parent. For example, the CUSTOMER class and the EMPLOYEE class share a parent PERSON class.
- [Inheritance](#) is the ability of an object within the class hierarchy to inherit the attributes and methods of the classes above it. For example, two classes, CUSTOMER and EMPLOYEE, can be created as subclasses from the class PERSON. In this case, CUSTOMER and EMPLOYEE will inherit all attributes and methods from PERSON.
- Object-oriented data models are typically depicted using [Unified Modeling Language \(UML\)](#) class diagrams. UML is a language based on OO concepts that describes a set of diagrams and symbols you can use to graphically model a system.

Object/Relational

Facing the demand to support more complex data representations, the relational model’s main vendors evolved the model further and created the [extended relational data model \(ERDM\)](#). The ERDM adds many of the OO model’s features within the inherently simpler relational database structure. The ERDM gave birth to a new generation of relational databases that support OO features such as objects (encapsulated data and methods),

extensible data types based on classes, and inheritance. That's why a DBMS based on the ERDM is often described as an [object/relational database management system \(O/R DBMS\)](#). Today, most relational database products can be classified as object/relational, and they represent the dominant market share of OLTP and OLAP database applications. The success of the O/R DBMSs can be attributed to the model's conceptual simplicity, data integrity, easy-to-use query language, high transaction performance, high availability, security, scalability, and expandability. In contrast, the OO DBMS is popular in niche markets such as computer-aided drawing/computer-aided manufacturing (CAD/ CAM), geographic information systems (GIS), telecommunications, and multimedia, which require support for more complex objects.

Emerging Data Models: Big Data and NoSQL

Deriving usable business information from the mountains of web data that organizations have accumulated over the years has become an imperative need. Web data in the form of browsing patterns, purchasing histories, customer preferences, behaviour patterns, and social media data from sources such as Facebook, Twitter, and LinkedIn have inundated organizations with combinations of structured and unstructured data.

The amount of data being collected grows exponentially every day. According to IBM, "Every day we create 2.5 quintillion bytes of data—so much that 90 percent of the data in the world today has been created in the last two years alone."

[Big Data](#) refers to a movement to find new and better ways to manage large amounts of web- and sensor-generated data and derive business insight from it, while simultaneously providing high performance and scalability at a reasonable cost.

The term seems to have been first used in a computing framework by John Mashey, a Silicon Graphics scientist in the 1990s.³ However, it seems to be Douglas Laney, a data analyst from the Gartner Group, who first described the basic characteristics of Big Data databases: volume, velocity, and variety, or the [3 Vs](#).

- *Volume* refers to the amounts of data being stored. With the adoption and growth of the Internet and social media, companies have multiplied the ways to reach customers. Over the years, and with the benefit of technological advances, data for millions of e-transactions were being stored daily on company databases. Furthermore, organizations are using multiple technologies to interact with end users and those technologies are generating mountains of data. This ever-growing volume of data quickly reached petabytes in size, and it's still growing.
- *Velocity* refers not only to the speed with which data grows but also to the need to process this data quickly in order to generate information and insight. With the advent of the Internet and social media, business response times have shrunk considerably. Organizations need not only to store large volumes of quickly accumulating data but also need to process such data quickly. The velocity of data growth is also due to the increase in the number of different data streams from which data is being piped to the organization (via the web, e-commerce, Tweets, Facebook posts, emails, sensors, GPS, and so on).
- *Variety* refers to the fact that the data being collected comes in multiple different data formats. A great portion of these data comes in formats not suitable to be handled by the typical operational databases based on the relational model.

The problem is that the relational approach does not always match the needs of organizations with Big Data challenges.

- It is not always possible to fit unstructured, social media and sensor-generated data into the conventional relational structure of rows and columns.
- Adding millions of rows of multiformat (structured and nonstructured) data on a daily basis will inevitably lead to the need for more storage, processing power, and sophisticated data analysis tools that may not be available in the relational environment
- Data analysis based on OLAP tools has proven to be very successful in relational environments with highly structured data. However, mining for usable data in the vast amounts of unstructured data collected from web sources requires a different approach.

There is no “one-size-fits-all” cure to data management needs (although many established database vendors will probably try to sell you on the idea). The business world has many examples of companies that leverage technology to gain a competitive advantage, and others that miss it. Just ask yourself how the business landscape would be different if:

- Blackberry had responded quickly to the emerging Apple smartphone technology.
- MySpace had responded to Facebook’s challenge in time.
- Blockbuster had reacted to the Netflix business model sooner.
- Barnes & Noble had developed a viable Internet strategy before Amazon.

In order to create value from their previously unused Big Data stores, companies are using new Big Data technologies. These emerging technologies allow organizations to process massive data stores of multiple formats in cost-effective ways. Some of the most frequently used Big Data technologies are Hadoop, MapReduce, and NoSQL databases.

- **Hadoop** is a Java-based, open-source, high-speed, fault-tolerant distributed storage and computational framework. Hadoop uses low-cost hardware to create clusters of thousands of computer nodes to store and process data. Hadoop originated from Google’s work on distributed file systems and parallel processing and is currently supported by the Apache Software Foundation.⁵ Hadoop has several modules, but the two main components are Hadoop Distributed File System (HDFS) and MapReduce.

Hadoop Distributed File System (HDFS) is a highly distributed, fault-tolerant file storage system designed to manage large amounts of data at high speeds. In order to achieve high throughput, HDFS uses the write-once, read many model. This means that once the data is written, it cannot be modified. HDFS uses three types of nodes: a **name node** that stores all the metadata about the file system, a **data node** that stores fixed-size data blocks (that could be replicated to other data nodes), and a **client node** that acts as the interface between the user application and the HDFS.

- **MapReduce** is an open-source application programming interface (API) that provides fast data analytics services. MapReduce distributes the processing of the data among thousands of nodes in parallel. MapReduce works with structured and non-structured data. The MapReduce framework provides two main functions: Map and Reduce. In general terms, the Map function takes a job and divides it into smaller units of work, and the Reduce function collects all the output results generated from the nodes and integrates them into a single result set. Although MapReduce itself is viewed as fairly limited today, it defined the paradigm for how Big Data is processed.

- **NoSQL** is a large-scale distributed database system that stores structured and unstructured data in efficient ways.

General characteristics of NOSQL databases:

- They are not based on the relational model and SQL; hence the name NoSQL.
- They support highly distributed database architectures.
- They provide high scalability, high availability, and fault tolerance.
- They support very large amounts of sparse data (data with a large number of attributes but where the actual number of data instances is low).
- They are geared toward performance rather than transaction consistency.